

Universidad Sergio Arboleda
Facultad de Ingeniería

Informe de Laboratorio

Taller Práctico 1: Filtros de Imagen con CUDA
Guillermo Andres De Mendoza Corrales

Antony Camilo Barahona Peña

Febrero 2026

Resumen

En este informe se documenta el desarrollo e implementación de cinco filtros de procesamiento digital de imágenes utilizando la tecnología CUDA a través de la librería Numba en Python. Cada algoritmo fue diseñado para ejecutarse en GPU mediante kernels paralelos, aprovechando la capacidad de procesamiento masivo que ofrece la arquitectura CUDA. Los filtros implementados fueron: inversión de colores, desenfoque, recorte, binarización y detección de bordes mediante el operador Sobel.

Índice

1. Introducción	3
2. Entorno de Desarrollo	3
3. Estructura General de los Algoritmos	3
4. Filtro 1: Inversión de Colores (Negativo)	4
4.1. Descripción	4
4.2. Fundamento Matemático	4
4.3. Implementación	4
4.4. Análisis	5
5. Filtro 2: Desenfoque (Blur)	5
5.1. Descripción	5
5.2. Fundamento Matemático	5
5.3. Implementación	5
5.4. Análisis	6
6. Filtro 3: Recorte de Imagen (Crop)	6
6.1. Descripción	6
6.2. Fundamento Matemático	6
6.3. Implementación	6
6.4. Manejo de Límites	7
6.5. Análisis	7
7. Filtro 4: Binarización (Imagen Blanco y Negro)	7
7.1. Descripción	7
7.2. Fundamento Matemático	7
7.3. Implementación	7
7.4. Análisis	8
8. Filtro 5: Detección de Bordes (Operador Sobel)	8
8.1. Descripción	8
8.2. Fundamento Matemático	8
8.2.1. Conversión a Escala de Grises	8
8.2.2. Kernels Sobel	8
8.2.3. Magnitud del Gradiente	9
8.3. Implementación	9
8.4. Análisis	9
9. Conclusiones	10

1. Introducción

El procesamiento de imágenes es una de las áreas donde la computación paralela en GPU demuestra sus mayores ventajas. Una imagen puede verse como una matriz bidimensional donde cada elemento (píxel) puede procesarse de forma independiente, lo que la convierte en un problema naturalmente paralelizable.

CUDA (Compute Unified Device Architecture) es la plataforma de computación paralela desarrollada por NVIDIA que permite ejecutar código directamente en la GPU. En este taller se utilizó Numba, una librería de Python que permite escribir funciones decoradas con `@cuda.jit` para compilarlas y ejecutarlas en la GPU sin necesidad de escribir código C/C++ directamente.

El objetivo principal del taller fue implementar cinco filtros distintos, cada uno abordando un tipo diferente de operación sobre imágenes: desde operaciones puntuales (que solo dependen del píxel mismo) hasta operaciones de vecindad (que dependen de los píxeles circundantes).

2. Entorno de Desarrollo

El desarrollo se realizó en Google Colaboratory con aceleración de hardware GPU tipo T4. El entorno utilizado fue:

- Python 3
- Numba (con soporte CUDA)
- NumPy
- OpenCV (cv2)
- Matplotlib

Para habilitar la compatibilidad entre versiones de Numba y CUDA se incluyó la siguiente configuración al inicio del notebook:

```
1 from numba import config
2 config.CUDA_ENABLE_PYNVJITLINK = 1
```

3. Estructura General de los Algoritmos

Todos los filtros implementados siguen la misma estructura de dos componentes:

1. **Kernel CUDA:** función decorada con `@cuda.jit` que se ejecuta en la GPU. Cada hilo del kernel procesa un píxel específico de la imagen, identificado por sus coordenadas (`i, j`).
2. **GPU Caller:** función Python que prepara los datos, configura la malla de hilos (grid y bloques), lanza el kernel y recupera el resultado de la GPU.

La configuración del grid utilizada en todos los ejercicios fue de bloques de 16×16 hilos:

```
1 threadsperblock = (16, 16)
2 blockspergrid_x = (COLUMNS + threadsperblock[0] - 1) //
  threadsperblock[0]
3 blockspergrid_y = (ROWS + threadsperblock[1] - 1) //
  threadsperblock[1]
4 blockspergrid = (blockspergrid_x, blockspergrid_y)
```

Esta fórmula garantiza que el grid sea siempre lo suficientemente grande para cubrir todos los píxeles, incluso cuando las dimensiones de la imagen no son múltiplos exactos de 16.

Las imágenes se cargan normalizadas en el rango $[0,0, 1,0]$ dividiéndolas entre 255, por lo que las operaciones matemáticas dentro de los kernels trabajan en este rango.

4. Filtro 1: Inversión de Colores (Negativo)

4.1. Descripción

La inversión de colores es una operación puntual, lo que significa que el nuevo valor de cada píxel depende únicamente de su propio valor original. Simula el efecto de un negativo fotográfico: los colores claros se vuelven oscuros y viceversa.

4.2. Fundamento Matemático

Para cada canal de color (R, G, B) de un píxel, la fórmula es:

$$C_{negativo} = 255 - C_{original} \quad (1)$$

Como la imagen está normalizada en $[0,0, 1,0]$, la fórmula equivalente es:

$$C_{negativo} = 1,0 - C_{original} \quad (2)$$

4.3. Implementación

```
1 @cuda.jit
2 def invert_kernel_2D(img_color, img_inverted):
3     j, i = cuda.grid(2)
4     rows = img_inverted.shape[0]
5     cols = img_inverted.shape[1]
6     if i < rows and j < cols:
7         img_inverted[i, j, 0] = 1.0 - img_color[i, j, 0]
8         img_inverted[i, j, 1] = 1.0 - img_color[i, j, 1]
9         img_inverted[i, j, 2] = 1.0 - img_color[i, j, 2]
```

4.4. Análisis

Al ser una operación puramente puntual, cada hilo trabaja de forma completamente independiente, lo que la convierte en el caso ideal de paralelismo en GPU. No hay dependencia entre píxeles ni accesos a memoria compartida.

5. Filtro 2: Desenfoque (Blur)

5.1. Descripción

El desenfoque suaviza la imagen eliminando ruido de alta frecuencia. A diferencia de la inversión, es una operación de vecindad: el nuevo valor de un píxel depende de sus píxeles circundantes. Se implementó un filtro de media (box filter), donde todos los vecinos tienen el mismo peso.

5.2. Fundamento Matemático

Para un kernel de tamaño $N \times N$ con radio $r = N \div 2$, el valor de salida de un píxel en la posición (i, j) es:

$$P_{out}(i, j) = \frac{1}{count} \sum_{di=-r}^r \sum_{dj=-r}^r P_{in}(i + di, j + dj) \quad (3)$$

donde *count* es el número de vecinos válidos dentro de los límites de la imagen.

5.3. Implementación

```
1 @cuda.jit
2 def blur_kernel_2D(img_color, img_blurred, kernel_size):
3     j, i = cuda.grid(2)
4     rows = img_color.shape[0]
5     cols = img_color.shape[1]
6     if i < rows and j < cols:
7         r = kernel_size // 2
8         total_r = 0.0
9         total_g = 0.0
10        total_b = 0.0
11        count = 0
12        for di in range(-r, r + 1):
13            for dj in range(-r, r + 1):
14                ni = i + di
15                nj = j + dj
16                if ni >= 0 and ni < rows and nj >= 0 and nj < cols:
17                    total_r += img_color[ni, nj, 0]
18                    total_g += img_color[ni, nj, 1]
19                    total_b += img_color[ni, nj, 2]
20                count += 1
```

```

21         img_blurred[i, j, 0] = total_r / count
22         img_blurred[i, j, 1] = total_g / count
23         img_blurred[i, j, 2] = total_b / count

```

5.4. Análisis

El tamaño del kernel es configurable por el usuario. Un valor de `kernel_size=3` produce un desenfoque leve, mientras que valores más altos como 9 o 15 generan un efecto más pronunciado. El manejo de bordes descarta los vecinos que caen fuera de la imagen y ajusta el divisor (`count`) dinámicamente para evitar distorsión en los márgenes.

6. Filtro 3: Recorte de Imagen (Crop)

6.1. Descripción

El recorte extrae una subregión rectangular de la imagen original. El usuario define el punto de inicio (`x_start`, `y_start`) y las dimensiones del recorte (`width_crop`, `height_crop`). La malla de hilos CUDA se lanza con el tamaño de la imagen resultante, no de la original.

6.2. Fundamento Matemático

La relación de mapeo entre las coordenadas del píxel en la imagen recortada (i, j) y su posición en la imagen original es:

$$(i_{original}, j_{original}) = (i + y_start, j + x_start) \quad (4)$$

6.3. Implementación

```

1 @cuda.jit
2 def crop_kernel_2D(img_color, img_cropped, x_start, y_start):
3     j, i = cuda.grid(2)
4     rows = img_cropped.shape[0]
5     cols = img_cropped.shape[1]
6     if i < rows and j < cols:
7         src_i = i + y_start
8         src_j = j + x_start
9         img_cropped[i, j, 0] = img_color[src_i, src_j, 0]
10        img_cropped[i, j, 1] = img_color[src_i, src_j, 1]
11        img_cropped[i, j, 2] = img_color[src_i, src_j, 2]

```

6.4. Manejo de Límites

Para evitar accesos fuera de memoria en la GPU, el caller valida y ajusta los parámetros antes de lanzar el kernel:

```
1 x_start = max(0, min(x_start, COLUMNS - 1))
2 y_start = max(0, min(y_start, ROWS - 1))
3 width_crop = min(width_crop, COLUMNS - x_start)
4 height_crop = min(height_crop, ROWS - y_start)
```

6.5. Análisis

Este filtro muestra una característica importante de CUDA: el grid no tiene que coincidir con el tamaño de la imagen original. Al lanzarlo con las dimensiones del recorte, cada hilo tiene una responsabilidad directa y no se desperdician recursos procesando píxeles que no pertenecen a la región de interés.

7. Filtro 4: Binarización (Imagen Blanco y Negro)

7.1. Descripción

La binarización convierte cada píxel a blanco (1.0) o negro (0.0) dependiendo de si su intensidad promedio supera un umbral T definido por el usuario.

7.2. Fundamento Matemático

Primero se calcula la intensidad promedio del píxel a partir de sus tres canales:

$$I = \frac{R + G + B}{3} \quad (5)$$

Luego se aplica la regla de decisión:

$$P_{out} = \begin{cases} 1,0 & \text{si } I > T_{norm} \\ 0,0 & \text{si } I \leq T_{norm} \end{cases} \quad (6)$$

donde $T_{norm} = T/255$ es el umbral normalizado al rango $[0,0, 1,0]$.

7.3. Implementación

```
1 @cuda.jit
2 def binary_kernel_2D(img_color, img_binary, threshold):
3     j, i = cuda.grid(2)
4     rows = img_binary.shape[0]
5     cols = img_binary.shape[1]
```

```

6     if i < rows and j < cols:
7         R = img_color[i, j, 0]
8         G = img_color[i, j, 1]
9         B = img_color[i, j, 2]
10        intensity = (R + G + B) / 3.0
11        if intensity > threshold:
12            img_binary[i, j] = 1.0
13        else:
14            img_binary[i, j] = 0.0

```

7.4. Análisis

El umbral se ingresa en la escala $[0, 255]$ desde el lado del CPU y se normaliza antes de pasarlo al kernel. Un valor de $T = 128$ produce un resultado equilibrado; valores bajos generan imágenes mayormente blancas y valores altos generan imágenes mayormente negras. La imagen de salida tiene un solo canal, por lo que se visualiza con `cmap='gray'`.

8. Filtro 5: Detección de Bordes (Operador Sobel)

8.1. Descripción

La detección de bordes identifica las zonas de cambio brusco de intensidad en una imagen. El operador Sobel estima el gradiente de intensidad en dos direcciones: horizontal (G_x) y vertical (G_y). Este filtro requiere dos pasos: primero convertir la imagen a escala de grises y luego aplicar la convolución Sobel.

8.2. Fundamento Matemático

8.2.1. Conversión a Escala de Grises

$$I_{gray} = 0,2989 \cdot R + 0,5870 \cdot G + 0,1140 \cdot B \quad (7)$$

8.2.2. Kernels Sobel

Los dos kernels 3×3 del operador Sobel son:

$$K_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \quad (8)$$

8.2.3. Magnitud del Gradiente

$$G = \sqrt{G_x^2 + G_y^2} \quad (9)$$

El cálculo de la raíz cuadrada se realiza con `cuda.libdevice.sqrtf()`, el método optimizado de CUDA para operaciones de punto flotante en GPU.

8.3. Implementación

Kernel de escala de grises:

```

1 @cuda.jit
2 def grayscale_for_sobel_kernel(img_color, img_gray):
3     j, i = cuda.grid(2)
4     rows = img_gray.shape[0]
5     cols = img_gray.shape[1]
6     if i < rows and j < cols:
7         R = img_color[i, j, 0]
8         G = img_color[i, j, 1]
9         B = img_color[i, j, 2]
10        img_gray[i, j] = 0.2989 * R + 0.5870 * G + 0.1140 * B

```

Kernel Sobel:

```

1 @cuda.jit
2 def sobel_kernel_2D(img_gray, img_edges):
3     j, i = cuda.grid(2)
4     rows = img_gray.shape[0]
5     cols = img_gray.shape[1]
6     if i > 0 and i < rows - 1 and j > 0 and j < cols - 1:
7         Gx = (img_gray[i-1, j+1] + 2.0 * img_gray[i, j+1] +
8              img_gray[i+1, j+1]) - \
9              (img_gray[i-1, j-1] + 2.0 * img_gray[i, j-1] +
10              img_gray[i+1, j-1])
11         Gy = (img_gray[i+1, j-1] + 2.0 * img_gray[i+1, j] +
12              img_gray[i+1, j+1]) - \
13              (img_gray[i-1, j-1] + 2.0 * img_gray[i-1, j] +
14              img_gray[i-1, j+1])
15         G = cuda.libdevice.sqrtf(Gx * Gx + Gy * Gy)
16         if G > 1.0:
17             G = 1.0
18         img_edges[i, j] = G

```

8.4. Análisis

Este fue el filtro más complejo del taller. Al requerir dos kernels en secuencia, fue necesario agregar `cuda.synchronize()` entre ambos lanzamientos para garantizar que el primer kernel haya terminado completamente antes de que el segundo comience a leer sus resultados. Los píxeles del borde de la imagen (primera y última fila/columna) quedan en cero ya que no tienen vecinos completos para aplicar la convolución.

9. Conclusiones

A lo largo de este taller se pudo comprobar en práctica cómo CUDA permite acelerar operaciones sobre imágenes distribuyendo el trabajo entre miles de hilos que actúan en paralelo. Algunos puntos clave aprendidos:

- El chequeo de límites dentro del kernel es fundamental para evitar accesos ilegales a memoria en la GPU.
- La normalización de la imagen al rango $[0,0, 1,0]$ simplifica las operaciones matemáticas.
- Cuando se encadenan múltiples kernels, `cuda.synchronize()` garantiza la consistencia de los datos entre pasos.
- La flexibilidad de pasar parámetros al kernel (como el tamaño del blur o el umbral de binarización) permite reutilizar el mismo kernel para distintos casos de uso sin recompilar.